

Week 4 · Lesson 4 — PROJECT: Deploy a React App to the ESP32

Time: ~5–6h · **Goal:** Prove the entire pipeline end-to-end: build a tiny React app, gzip it, flash it to the device, and load it in a browser from the EVB-01. **This is the most important milestone of the whole plan** — once this works, the rest is "just build the screens."

Why this matters

Everything before this was practice. This lesson connects React → Vite → gzip → SPIFFS → the live device. It's also the step most likely to fight you, so do it on a *throwaway* app now, before you've built the real dashboard. Expect to pair with your manager here — firmware edits are collaborative.

The pipeline (do it in order)

1. **Build** your Vite React app: `npm run build` → `dist/` (with `base: './'` and the gzip plugin from Lesson 3, so `.gz` files are emitted).
2. **Place the files in** `data/`. Decide a layout, e.g. `data/index.html.gz` and `data/assets/*.gz`. Keep **only** the `.gz` files to save flash.
3. **Serve them from the firmware.** Because `serveStatic` won't set `Content-Encoding` on the ESP32 (Lesson 3), add custom handlers in `web_server.cpp` that open each `.gz` and send `Content-Encoding: gzip`. With your manager, decide whether to:
 - register a handler per asset, or
 - write a small generic handler that maps any request path to `<path>.gz` and sets the header by file extension.Also make sure the **root** `/` serves your gzipped `index.html`, and ideally that unknown routes fall back to it (so the SPA loads).
4. **Flash the filesystem:** `pio run -t uploadfs` (or your `custom_upload` shortcut).
5. **Open the device's IP/hostname in a browser** and confirm your React app loads — check the **Network tab** in DevTools: each asset should show `Content-Encoding: gzip` and a `200`, with no gibberish in the response.

Watch out for (the usual failure modes)

- **Gibberish / blank page** → you served a `.gz` without the `Content-Encoding: gzip` header. Fix the handler.
- **404 on `/assets/ ...`** → path mismatch between Vite's output and your firmware routes. This is the `serveStatic` gotcha; align the paths or use the generic handler. (Set `base: './'`.)
- **MIME type warnings** → set the correct content type (`text/html` , `application/javascript` , `text/css`) when streaming.
- **Won't fit** → check `dist/` gzipped total against the ~1.5MB budget; re-check your Vite build isn't bundling something huge.

Reference

- Your Lesson 3 gzip notes + the ESP32 `.gz` issue link.
- The existing `serveStatic( ... )` routes in `web_server.cpp` (read them first to see the current pattern).
- Random Nerd — *PlatformIO Upload Files to SPIFFS*: <https://randomnerdtutorials.com/esp32-vs-code-platformio-spiiffs/>
- Vite — *base* config: <https://vite.dev/config/>

Deliverable → push to git

1. On a `feature/react-ui-spike` branch of the EVB-01 repo: the tiny React app's `dist` deployed + the firmware handler changes.
2. A screenshot of DevTools Network tab showing `Content-Encoding: gzip` on your assets, and the app rendering from the device.
3. Commit: `git commit -m "Week 4 MILESTONE: React app builds, gzips, and serves from the EVB-01"` and push.
4. **Show your manager — this is the GO/NO-GO checkpoint.** If you're stuck after a focused attempt, ask for help here; don't grind for days.

Definition of done

- [] A React app loads in a browser **from the device** (not just `npm run dev`).
- [] Assets are served gzipped with the correct `Content-Encoding` header.
- [] Total flashed UI fits the filesystem budget.
- [] You can repeat the build→gzip→uploadfs cycle from memory.